



DesignWare DW_memctl Driver Kit User Guide

DesignWare Synthesizable Components for AMBA 2
DW_memctl

Version 1.00c

January 2011

Copyright Notice and Proprietary Information

Copyright © 2011 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Registered Trademarks (®)

Synopsys, AEON, AMPS, Astro, Behavior Extracting Synthesis Technology, Cadabra, CATS, CRITIC, Certify, CHIPit, Confirma, CODE V, Design Compiler, DesignWare, EMBED-IT!, Formality, Global Synthesis, HDL Analyst, HSIM, HSPICE, Identify, Leda, LightTools, MAST, ModelTools, NanoSim, NOVeA, OpenVera, ORA, PathMill, Physical Compiler, PrimeTime, SCOPE, Simply Better Results, SiVL, SNUG, SolvNet, Sonic Focus, STAR Memory System, Syndicated, Synplicity, the Synplicity Logo, Synplify, Synplify Pro, Synthesis Constraints Optimization Environment, TetraMAX, UMRBus, VCS, Vera, and YIELDirector are registered trademarks of Synopsys, Inc.

Trademarks (™)

AFGen, Apollo, ARC, ASAP, Astro, Astro-Rail, Astro-Xtalk, Aurora, AvanWaves, BEST, Columbia, Columbia-CE, Cosmos, CosmosLE, CosmosScope, CRITIC, CustomExplorer, CustomSim, DC Expert, DC Professional, DC Ultra, Design Analyzer, Design Vision, DesignerHDL, DesignPower, DFTMAX, Direct Silicon Access, Discovery, Eclipse, Encore, EPIC, Galaxy, Galaxy Custom Designer, HANEX, HAPS, HapsTrak, HDL Compiler, Hercules, Hierarchical Optimization Technology, High-performance ASIC Prototyping System, HSIMplus, i-Virtual Stepper, IICE, in-Sync, iN-Tandem, Intelli, Jupiter, Jupiter-DP, JupiterXT, JupiterXT-ASIC, Liberty, Libra-Passport, Library Compiler, Macro-PLUS, Magellan, Mars, Mars-Rail, Mars-Xtalk, Milkyway, ModelSource, Module Compiler, MultiPoint, ORAengineering, Physical Analyst, Planet, Planet-PL, Polaris, Power Compiler, Raphael, RippledMixer, Saturn, Scirocco, Scirocco-i, SiWare, Star-RCXT, Star-SimXT, StarRC, Taurus, TotalRecall, TSUPREM-4, VCSi, VHDL Compiler, VMC, and Worksheet Buffer are trademarks of Synopsys, Inc.

Service Marks (SM)

MAP-in, SVP Café, and TAP-in are service marks of Synopsys, Inc.

SystemC is a trademark of the Open SystemC Initiative and is used under license.

ARM and AMBA are registered trademarks of ARM Limited.

Saber is a registered trademark of SabreMark Limited Partnership and is used under license.

PCI Express is a trademark of PCI-SIG.

All other product or company names may be trademarks of their respective owners.

Synopsys, Inc.
700 E. Middlefield Road
Mountain View, CA 94043
www.synopsys.com

Contents

Chapter 1

Installation and Setup	3
Downloading the Release Image	3
Installing the Release Image	3
Installing the DW_memctl Driver Kit	4
Checking Hardware/OS Requirements	4
Checking Software Tool Requirements	4
Getting and Setting Up ARM-946 CPU Model	5
Setting Up Your Environment	5

Chapter 2

Release Notes	7
Installation and Configuration	7
New Features and Changes	7
STARs on the Web	8
Fixed Problems/Enhancements	8
Known Problems and Workarounds	8
Late Breaking Information	9
Getting Help	9

Chapter 3

Using the DW_memctl Driver Kit	11
Overview	11
Definitions	12
Public Header File	13
Private Header File	13
Common Header Files	13
Naming Conventions	13
Application Programming Interface	14
DW_memctl Driver Kit Directory	15
Driver Kit Usage	15

Chapter 4

DW_memctl Driver Kit Verification	19
Verification Strategy	19
FPGA Testing	19
Unit Tests	19

1

Installation and Setup

This chapter provides detailed information on how to install and set up the DW_memctl Driver Kit. The following checklist summarizes the activities you need to perform:

1. “Downloading the Release Image”
2. “Installing the Release Image”
3. “Installing the DW_memctl Driver Kit” on page 4
4. “Checking Hardware/OS Requirements” on page 4
5. “Checking Software Tool Requirements” on page 4
6. “Getting and Setting Up ARM-946 CPU Model” on page 5
7. “Setting Up Your Environment” on page 5

For more information about the DW_memctl Driver Kit and how to use it, refer to “Using the DW_memctl Driver Kit” on page 11.

Downloading the Release Image

The DW_memctl Driver Kit requires you to have the correct version of the DW_memctl synthesizable IP component, which was released with the current release. For version information, refer to “Supported Versions for Components and Tools” in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*. If you or your site administrator have not yet downloaded this release image, that is your first step.

If you have received e-mail from Synopsys informing you about the latest release, use the URL contained in the e-mail to locate the download page. Otherwise, you must first fill out a download request:

http://www.synopsys.com/dwrequest/forms/dwamba_form.html

Installing the Release Image

If you or your site administrator have already downloaded the release image, follow the procedure on the download page to unpack the release image on your local network.

For information about installing components or a related tools, refer to “Installation Instructions” in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

Installing the DW_memctl Driver Kit

You must install the DW_ahb_dmac Driver Kit independently from the current AMBA release. Download and installation instructions are included on the DW_memctl Driver Kit download page (sent to you in an e-mail from Synopsys). Otherwise, you must complete a download request on the following site:

http://www.synopsys.com/dwrequest/forms/dwamba_form.html



Attention

The DW_memctl Driver Kit requires the DWC-MEMCTL-Source license.

Checking Hardware/OS Requirements

For information about supported operating systems and platforms, refer to “[Supported Versions for Components and Tools](#)” in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

Checking Software Tool Requirements

The DW_memctl Driver Kit requires you to have the following tools:

- ARM-946 CPU model (to run the driver example) – you must obtain this simulation model from an ARM representative as described in “[Getting and Setting Up ARM-946 CPU Model](#)” on page 5.
- ARM compiler and related tools – go to the ARM web site to obtain information about their developer tools:

<http://www.arm.com/products/DevTools/>



Note

As stated previously, to use the DW_memctl driver example, you must have installed the current AMBA release so that you have the correct versions of the DW_memctl component. For version information, refer to “[Supported Versions for Components and Tools](#)” in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

- Vera
- VCS or MTI simulator



Note

There are no driver examples available for NC-Verilog simulators.

“[Supported Versions for Components and Tools](#)” in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide* lists the specific versions of these software tools that work with this version of the DW_memctl Driver Kit.

Getting and Setting Up ARM-946 CPU Model

To obtain the ARM-946 CPU simulation model from ARM:

1. Contact your ARM representative about obtaining ARM simulation models and compilers as Synopsys does not distribute these.
2. Ensure that ARMHOME environment variable has been set to point to ARM compiler directory, for example:

```
% setenv ARMHOME arm_installed_path/arm_ads_1.2
```

3. Ensure that ASMHOME environment variable has been set to point to the ARM-946 simulation model directory, for example:

```
% setenv ASMHOME arm_installed_path/ARM946E-S
```

For more information about running the DW_memctl driver example for the ARM-946 CPU model, refer to [“Example Application” on page 21](#).

Setting Up Your Environment

For the supported tools and version with this release of the DW_memctl Driver Kit, refer to [“Supported Versions for Components and Tools”](#) in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

2

Release Notes

This release note presents the latest information about the DesignWare DW_memctl Driver Kit, version 1.00c.

For complete information about the DW_memctl synthesizable IP component, refer to the [DesignWare DW_memctl Databook](#) and [DesignWare DW_memctl Release Notes](#).

This release note contains the following topics:

- [Installation and Configuration](#)
- [New Features and Changes](#)
- [STARs on the Web](#)
- [“Fixed Problems/Enhancements” on page 8](#)
- [“Known Problems and Workarounds” on page 8](#)
- [“Late Breaking Information” on page 9](#)
- [“Getting Help” on page 9](#)

Installation and Configuration

For installation instructions and required tools/versions/licenses, and for general release information, refer to [“Installation and Setup” on page 3](#).

New Features and Changes

The following change has been implemented in the 1.00c release, which is included in 2008.09a AMBA release:

- Updates to the license copyright notice in the driver source files

The following change has been implemented in the 1.00b release, which is included in 2007.06a AMBA release:

- In earlier versions, the DW_memctl.c file stated:

Please Unencrypt Driver from HomePage in dw_connect or coreConsultant to populate this file. Source license required.

Because dw_connect has been obsoleted and replaced with coreAssembler, the current version now states:

Please Unencrypt Driver from HomePage in coreAssembler or coreConsultant to populate this file. Source license required.

Table 1 lists the latest versions of the DW_memctl Driver Kit component.

Table 1: AMBA/DW_memctl Releases

DesignWare for AMBA 2 Release	DW_memctl Driver Kit Version	Databook Date
2008.09a	1.00c	September 2008
2007.06a	1.00b	June 2007
2005.04a	1.00a	April 2005

STARs on the Web

You can view a complete list of problem reports for this product, including problems identified after product release, by accessing the STAR report on the Web. Note that you must have a SolvNet ID in order to view STAR reports. For a list of STARs for the DW_memctl, go to the following link:

https://www.synopsys.com/dw/star.php?c=DW_memctl

You can access STAR reports for any DesignWare verification model or synthesizable component via the IP Directory:

<http://www.synopsys.com/designware/ipdir>

Fixed Problems/Enhancements

This section is reserved for future bug fixes and enhancements as there are no bug fixes or enhancements in this initial release of the product.

Known Problems and Workarounds

The following are known problems or workarounds for this release of the DW_memctl Driver Kit:

- Known issue in 1.00b version of DW_memctl Driver Kit – In coreAssembler/coreConsultant, clicking on “Unencrypt MEMCTL DriverKit Code” unencrypts the DW_memctl.c file, but you will see the following error in the cA/cC console:

```
Error: unknown command 'check_License' (CMD-005)
```

This error message can be ignored as it does not affect the generated output code.

- STAR 9000044349 – The DesignWare software drivers currently do not support a processor operating in big-endian mode. Specifically, there are some macros used for accessing device FIFOs, which do not function correctly for big-endian. These macros will need to be re-written to support both endian types.

Workaround: Use a processor in little-endian mode instead of big-endian mode.

Late Breaking Information

To see information that was not available when these release notes were created, please refer to:

http://www.synopsys.com/designware/docs/doc/amba/latest/dw_amba_200809.pdf

Getting Help

If you have a question about using Synopsys products, please consult product documentation that is installed on your network or located at the root level of your Synopsys product CD-ROM (if available). You can also access documentation for DesignWare products on the Web:

- Product documentation for many DesignWare products:

<http://www.synopsys.com/designware/docs>

- Datasheets for individual DesignWare IP components:

<http://www.synopsys.com/designware/ipdir>

You can also contact a Synopsys Support Center online or by phone:

- <http://www.synopsys.com/support/support.html>

- United States:

Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific Time, Mon—Fri.

- Canada:

Call 1-650-584-4200 from 7 AM to 5:30 PM Pacific Time, Mon—Fri.

- All other countries:

Find other local support center telephone numbers at the following URL:

http://www.synopsys.com/support/support_ctr

3

Using the DW_memctl Driver Kit

This appendix provides information about the DW_memctl Driver Kit, version 1.00a.

The topics are:

- [Overview](#)
- [“Definitions” on page 12](#)
- [“Naming Conventions” on page 13](#)
- [“Application Programming Interface” on page 14](#)
- [“DW_memctl Driver Kit Directory” on page 15](#)
- [“Driver Kit Usage” on page 15](#)

Overview

The DW_memctl Driver Kit allows you to easily program the DW_memctl component and integrate it into your higher-level application. It includes a set of application programming interfaces (APIs) and some example code.

Using the DW_memctl Driver Kit is the recommended way of programming the DW_memctl component. It provides the following benefits to IP designers:

- Proven method of access to DW_memctl minimizing usage errors
- Rapid software development with minimum overhead
- Detailed knowledge of DW_memctl register bit fields not required
- Easy integration of DW_memctl into existing software system
- Programming at register level eliminated

The Driver Kit is written in portable C (ANSI C99) and it is designed to work with any 32-bit embedded processor. In order to use the Driver Kit, it needs to be set up and integrated to either a specific operating system (OS) or an existing software environment. Setting up the Driver Kit means allocating memory for some internal data structures and also initializing the device data structure with required values, like device base address, device name, device type, APB data bus width, and so on. Integrating to an existing software infrastructure means associating a DW_memctl device structure with existing system or OS devices.

The Driver Kit is comprised of the following files:

- DW_memctl.c – C source code
- DW_memctl_public.h – Public header file
- DW_memctl_private.h – Private header file

To decrypt the DW_memctl Driver Kit, point to the DWC-MEMCTL-Source license and do the following:

1. Invoke coreConsultant or coreAssembler.
2. Click on “DesignWare for AMBA 2 and AXI” under “Synopsys DesignWare Links.”
3. Click on “DriverKits.”
4. Click on “Overview and Instructions” at the bottom of the page.
5. On the DesignWare IP Software DriverKit page, there is an “Unencrypt MEMCTL DriverKit Code” link. Click on that link and it will decrypt the .c file and place it in \$DESIGNWARE_HOME/drivers/DW_memctl/latest/ dir.

Definitions

The following terms relate to the DW_memctl Driver Kit:

API (Application Program Interface)

Generic term which refers to one or more related group of functions which provide a programming interface of some kind. For example, there is the DW_memctl public API, of which its Command API is a subset.

Common Directory

Contains a number of headers files which are used by all the DesignWare synthesizable IP Driver Kits. They provide a standard set of macros and definitions.

Device Structure

Contains information about a specific DW_memctl device. This includes its physical base address, data bus width, RTL version, hardware parameters, and so on. All Driver Kit functions use this structure to operate on one or more DW_memctl devices within the same system.

Driver

Refers to the resultant software from the porting of a Driver Kit into an OS or existing software infrastructure.

Driver Kit

Refers to the software provided as part of the DW_memctl Driver Kit. It is necessary to customize this software in order for it to be a fully usable driver.

Hardware Parameters

Determined by the hardware engineer responsible for generating and instantiating a particular peripheral. They cannot be changed after synthesis, and the relevant Driver Kits need to be informed of the chosen parameters. DW_memctl version 2.71 and earlier require you to specify the hardware parameter settings in the dw_memctl_param structure before using the Driver Kit. From version 2.72 and later, identification registers are included in the DW_memctl memory map. This removes the need for you to specify the hardware parameter settings of a DW_memctl device. Instead, the Driver Kit automatically discovers these settings during initialization. For more information, refer to “[Registers](#)” in the *DesignWare DW_memctl Databook*.

Private Header File

Contains implementation-specific functions and data types that are used internally. It is not necessary to include this file to program the DW_memctl. The only exception to this is when the DW_memctl device structure is being created during Driver Kit setup.

Public Header File

Contains the public interface (API) for the DW_memctl Driver Kit. The functions and data types therein provide full control over the programming of a DW_memctl device.

Public Header File

The public API definitions for the Driver Kit are found in DW_memctl_public.h and should be included by any file that uses this Driver Kit.

Private Header File

The private header file, DW_memctl_private.h, contains data types and functions that are used internally by the Driver Kit. Normally, this header file should not be included by any user files that access the API functions as implementation details are subject to change without notice. It is best thought of and treated as the private part of a C++ class. The only time one needs to include the file is during Driver Kit setup, when providing memory for and initializing internal data structures. The public header must be included prior to including the private header to prevent compile errors.

Common Header Files

As part of the Driver Kit package, common header files are provided. These headers provide common data types and utility macros that are shared by all DesignWare synthesizable IP Driver Kits. The common headers are comprised of the following files:

- DW_common.h – includes all common headers as well as some standard C headers
- DW_common_bitops.h – bit masking and manipulation macros
- DW_common_dbc.h – assertion macros used for testing and debugging
- DW_common_errno.h – definitions of error codes that functions return
- DW_common_io.h – low-level input/output access macros for various data bus endianness and widths
- DW_common_types.h – various data type definitions and structures
- DW_common_list.h – macro support for linked lists

Naming Conventions

- All DW_memctl functions and data types begin with the prefix dw_memctl.
- All DW_memctl enumerated data types begin with Memctl_.
- All enumerated data types begin with a single capitalized letter.
- All macros are defined in upper case.

Application Programming Interface

The DW_memctl Driver Kit public API can be broadly divided into the following categories:

Data Types

Enumerated and defined constants that provide abstraction of input arguments and return values for Driver Kit functions.

Configuration API

Functions that configure a DW_memctl device in preparation for some functional operation(s) to be performed. There is minimal effect on the DW_memctl hardware device when calling a configuration function. Configuration registers are changed, but no functional operation is started or terminated by the device. Typically, this configuration must be performed before using DW_memctl.

Command API

These functions cause some functional behavior within the DW_memctl device to begin or halt. Examples of this would be functions which cause DW_memctl force a memory into deep-power-down mode, or to update the mode register.

Status API

These functions return status information about the current state of the device and/or the Driver Kit software. This is typically a direct register read from the DW_memctl memory map.

Figure 1 illustrates the DW_memctl Driver Kit at a conceptual level. In this figure, *Driver Kit* represents DW_memctl.c, *Common* represents the common headers files – that is, DW_common.h – and *User Code* represents initialization and setup code required to use the Driver Kit. For an implementation of User Code, refer to “[Example Application](#)” on page 21.

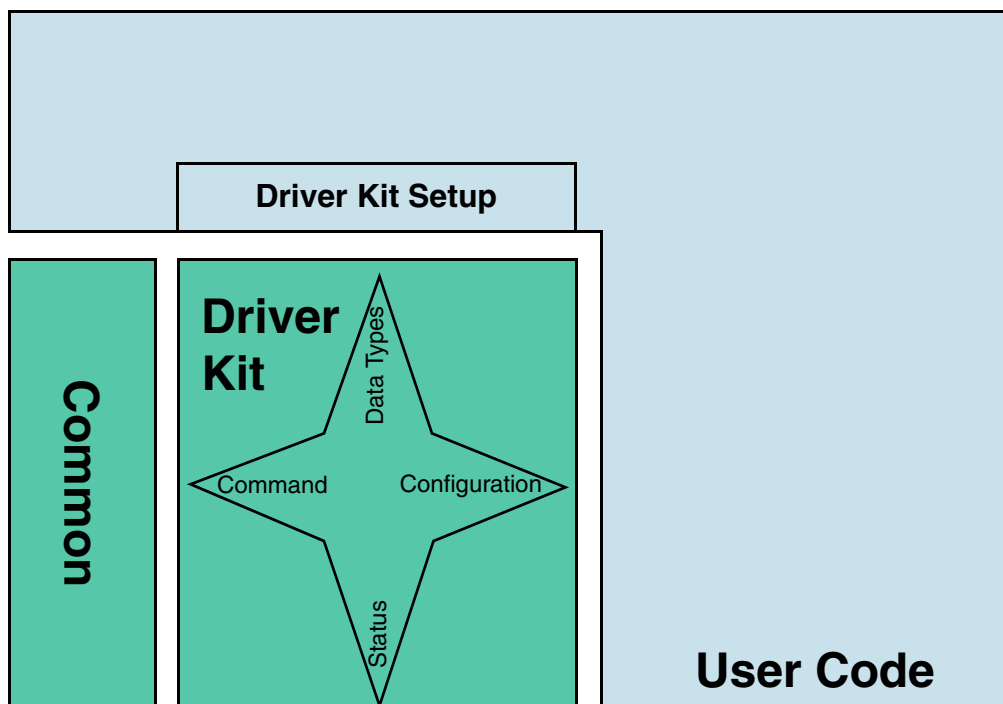


Figure 1: DW_memctl Driver Kit Block Diagram

The DW_memctl API functions are documented in DW_memctl.html, which resides in \$DESIGNWARE_HOME/drivers/DW_memctl/latest/doc. This file is automatically created from the public header file, DW_memctl_public.h.

DW_memctl Driver Kit Directory

The following table shows the directory contents of the DW_memctl Driver Kit. The user files are highlighted in bold.

Table 2: Directory of DW_memctl Driver Kit

Directory/File	Description
\$DESIGNWARE_HOME/drivers	Root directory for all Driver Kits
/DW_memctl	
/latest DW_memctl.c DW_memctl_public.h DW_memctl_private.h DW_memctl_example.c /doc dw_memctl_driver.pdf DW_memctl.html DW_memctl.css	Driver Kit code Public header Private header Driver Kit usage example This document API documentation Cascading Style Sheet for DW_memctl.html
/common /latest DW_common.h DW_common_bitops.h DW_common_dbc.h DW_common_errno.h DW_common_io.h DW_common_types.h DW_common_lists.h	All the common header files

Driver Kit Usage

The following is the typical usage flow for the Driver Kits:

1. Setting up the Driver Kit

Driver Kit setup can be performed within an operating system if one exists, otherwise, it can be performed by user code.

- a. Declare and initialize the dw_device structure. As with all DesignWare Driver Kits, the first argument to all the functions is a pointer of type dw_device to the DW_memctl device being programmed. The dw_device structure is defined in the common header files. The following are some notes about the structure members to be aware of:
 - **os* is a redundant pointer available for any purpose, associating the DesignWare Driver Kits with an operating system device handle. It is not used by the Driver Kit itself. It can be initialized to NULL if it is not being used.

- `uint32_t comp_version` is always hardwired inside the device and automatically initialized. It does not have to be manually initialized.
 - b. Declare the `dw_memctl_param` structure. This structure is defined in the private header file and can be manually initialized using the provided `DW_CC_DEFINE_MEMCTL_PARAMS` macro. The `dw_memctl_param` structure is automatically initialized by the Driver Kit if the DW_memctl RTL version is 2.72 or later. In this case, `dw_memctl_param` does not have to be manually initialized, although space must still be allocated for it. For more information, refer to “[Registers](#)” in the *DesignWare DW_memctl Databook*.
 - c. Initialize the device/Driver Kit with the `dw_memctl_init()` function. This is the first function that you must call before using the Driver Kit.
2. Configuring the device

You configure the DW_memctl device according to the requirements of a particular application in a given system using the configuration functions. Typically, a memory controlled by DW_memctl cannot be accessed until DW_memctl has been correctly configured to use it. Each memory associated with DW_memctl has its own chip select line. This chip select is specified for all the common API functions when setting/getting the base address, memory size, memory type, and so on.

There are three types of memory supported by DW_memctl: synchronous dynamic RAM (SDR-SDRAM, Mobile DDR-SDRAM, and so on), static memories, and SyncFlash memories. The API functions used for configuring each type of memory differ, as do the prefixes of these function names. The list of prefixes and memory types is:

- `dw_memctl_sd_*` – synchronous dynamic RAM
- `dw_memctl_sm_*` – static memory
- `dw_memctl_sf_*` – SyncFlash

If the power-on reset values of the DW_memctl registers have already been properly setup, there is no need to configure DW_memctl after the Driver Kit has been initialized ([Step 1](#)). In this case, the configuration step ([Step 2](#)) is optional. For more information, refer to the “[Parameters](#)” in the *DesignWare DW_memctl Databook*.

3. Accessing Memories

Once the DW_memctl has been correctly configured to control its associated memories, they are accessed in the normal way over the system bus. The address range of each memory is dictated by the various DW_memctl settings.

Figure 2 shows the usage flow diagram of the DW_memctl Driver Kit.

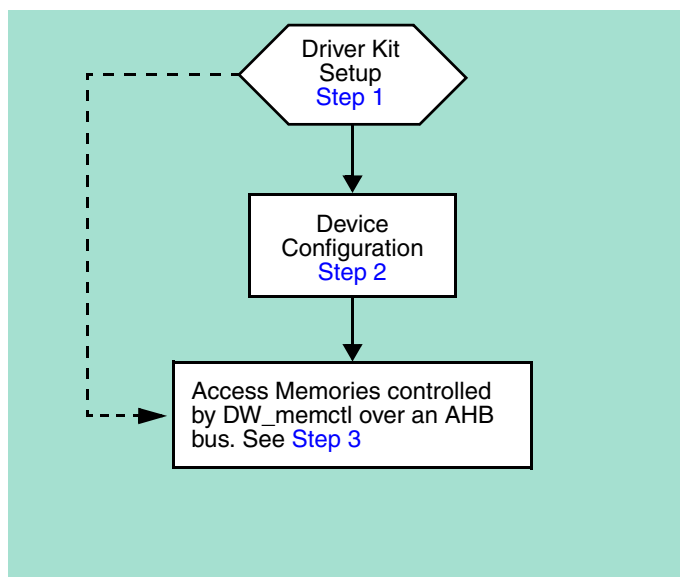


Figure 2: Usage Flow Diagram of DW_memctl Driver Kit

4

DW_memctl Driver Kit Verification

This chapter presents an overview of the verification of the DW_memctl Driver Kit.

Verification Strategy

A white-box testing strategy is employed throughout the development of the Driver Kit, using unit tests to cover each functional aspect of the Driver Kit. White-box testing assumes a detailed knowledge of the internal workings of the application (the box) and depends upon that knowledge. The principles of test-driven development and automated testing are also employed.

FPGA Testing

The Driver Kits are developed and tested utilizing an ARM Integrator platform coupled with an FPGA, which is loaded with a subsystem containing DesignWare peripherals with various configurations. As it is possible to synthesize several different DW_memctl devices by changing its hardware parameters, multiple FPGA images are used with the Driver Kit during regression testing.

Unit Tests

The DW_memctl Driver Kit regression test suite comprises a number of unit tests divided into test suites. The test harness consists of a DW_memctl connected to two static memories and an AHB data bus, all of which are connected to a processor. It is important to note that the tests are only seeking to verify the functionality of the Driver Kit and not the entire DW_memctl device. The unit tests are divided into the following categories:

Common API

This suite tests functions that are common to all memory types. Most unit tests are of the read/modify/write variety.

Synchronous Dynamic RAM API

This suite tests functions that are related to the dynamic RAM portion of the memory controller. Most unit tests are of the read/modify/write variety.

Static Memory API

This suite tests functions that are related to the static memory portion of the memory controller. Most unit tests are of the read/modify/write variety.

SyncFlash API

This suite tests functions that are related to the synchronous flash portion of the memory controller. Most unit tests are of the read/modify/write variety.

Device Functionality

This suite verifies the functional operation of DW_memctl. The memory controller is configured for two static memories, which are connected to it. After configuration, software performs extensive read/write tests across the entire address range of the static memories. If these tests pass, the Driver Kit is deemed to have correctly configured DW_memctl for normal function operation, using a marinescu algorithm.